

Day3 · 16차시

AI 기반 테스트 자동화

테스트를 쓰는 일도 AI에게 — 코드를 맡겼다면 검증도 맡긴다.
15차시에서 평가하던 내 서비스를, 이제 매번 자동으로 지킨다.

SECTION 00

오프닝 — 빨리 짤 코드, 누가 보증하나

평가는 한 번 보는 것 · 테스트는 매번 자동으로 보는 것

“AI가 코드를 빠르게 짠다 — 그런데 그 코드가 맞다는 건 누가 보장하나?”

— 15차시 ‘무엇이 좋은 답인가’ → 16차시 ‘그 기준을 자동으로 돌린다’

평가에서 자동화로

▶ 평가는 한 번 보는 스냅샷 · 테스트는 변경마다 자동으로 보는 감시

15차시 · 평가

- 무엇이 좋은 답인가를 정의
- 골든셋 · 루브릭 · 통과율
- 사람이 한 번 돌려 보는 스냅샷

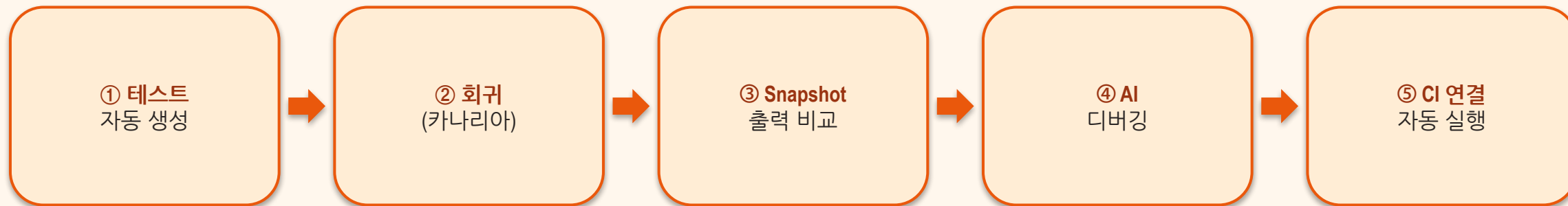
VS

16차시 · 테스트 자동화

- 그 기준을 코드로 굳힌다
- 변경할 때마다 자동 실행
- 같은 `my_service`를 매번 지킨다

이 차시 동안 얻는 것

▶ 다섯 블록이 이어지면 하나의 테스트 루프



SECTION 01

Codex로 테스트 생성 — 코드를 훑으면, 테스트도 맡겨라

테스트는 위임에 최적인 일

▶ 코드를 쓴 사람이 테스트도 쓰면 같은 사각지대를 공유한다 — AI = 다른 시선

사람이 할 일

- 무엇을 검증할지(의도)
- 엣지·예외의 정의
- 생성된 테스트 검토

VS

AI에 맡길 일

- 반복적인 assert 작성
- 정상 경로 케이스 나열
- 경계값 조합 채우기

테스트 생성 프롬프트의 3요소

▶ 좋은 테스트는 happy path를 넘어 희귀·잘못된 형식까지 망라

나쁜 요청

- “테스트 만들어줘”
- 대상·범위 불명확
- happy path만 대충

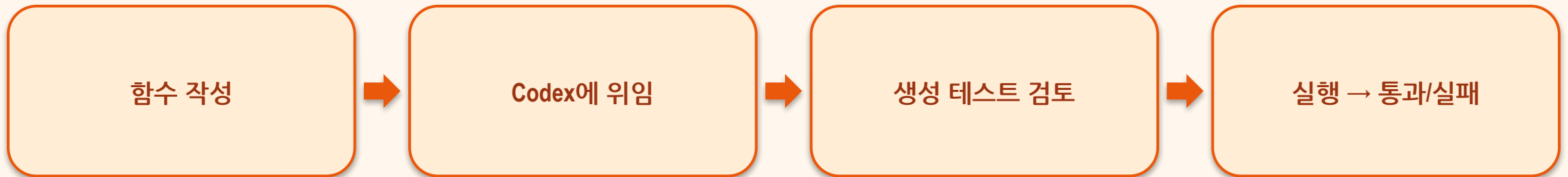
VS

좋은 요청

- ① 대상 함수/시그니처
- ② 정상 경로(happy path)
- ③ 엣지·예외(빈 입력·경계·오류)

코드 → 테스트 위임 흐름

▶ 종료 = 테스트가 모두 통과하고, 사람이 테스트 자체를 신뢰할 때



🔄 실패면 코드/테스트 수정 → 다시

테스트 생성, 이렇게 요청한다

▶ 함수 소스를 통째로 주고, 3요소를 명시 → `assert` 단위 테스트를 받는다

```
GEN = f"""다음 함수의 단위 테스트를 작성하라.  
{TARGET_SRC}  
- def test_...(): 안에서 assert 사용  
- 정상 경로 + 경계값(0, 최대) + 예외(ValueError)  
- 코드만 출력, 설명·펜스 없이"""  
# client.chat.completions.create(..., temperature=0) ← 재현성
```

생성된 테스트는 반드시 검토

▶ 테스트가 코드를 검증하지만, 그 테스트는 사람이 검증한다

왜

AI는 틀린 기대값도 그럴듯하게 쓴다 (예: 반올림을 거꾸로)

무엇을

생성 테스트를 사람이 한 번 읽고 의도와 맞는지 확인

안 하면

틀린 테스트로 틀린 코드를 보증하는 최악의 상황

요령

의심되면 버그를 일부러 심어 테스트가 잡는지 확인

SECTION 02

Regression — 프로덕션의 카나리아

잘 되던 게 조용히 깨지는 것

▶ 시소처럼 — 한쪽을 누르면 반대쪽이 뜨는데, 그쪽을 보는 사람이 없다

기대

- 새 기능 추가
- 기존 기능 그대로

VS

현실(회귀)

- 한쪽을 고쳤더니
- 다른 쪽이 **조용히** 깨짐
- 아무도 모르게 프로덕션으로

평가셋 = 살아있는 명세 (living spec)

▶ 고품질 평가셋은 에이전트가 처리할 것의 living specification

기본

입력 + 기대 결과를 모은다 — 변경마다 다시 돌려 회귀 조기 탐지

자란다

실패할 때마다 예제를 추가 — 평가셋이 스스로 크다

확장

로그 마이닝·모델 생성으로 실제 사용 패턴을 흡수

exact-match로는 회귀를 못 잡는다

▶ LLM은 추론 시에도 비결정적 — 거짓 경보만 양산

exact-match (취약)

- 출력 == 기대 비교
- LLM은 매번 다른 출력
- 사소한 표현 차이에도 깨짐

VS

구조/의미 비교 (견고)

- 의미 유사도(임베딩)
- 핵심 키워드 포함 여부
- 도구 호출·파라미터 일치

회귀를 잡는 비교의 축들

▶ 답(텍스트)만이 아니라 과정(도구·파라미터·구조)까지 — 한 예제로 여러 축을 동시에

의미 유사도

임베딩 거리 — 표현이 달라도 뜻이 같으면 통과

핵심 키워드

꼭 들어가야 할 문구의 포함 여부

도구 호출 일치

올바른 도구를 불렀는가

파라미터 정확도

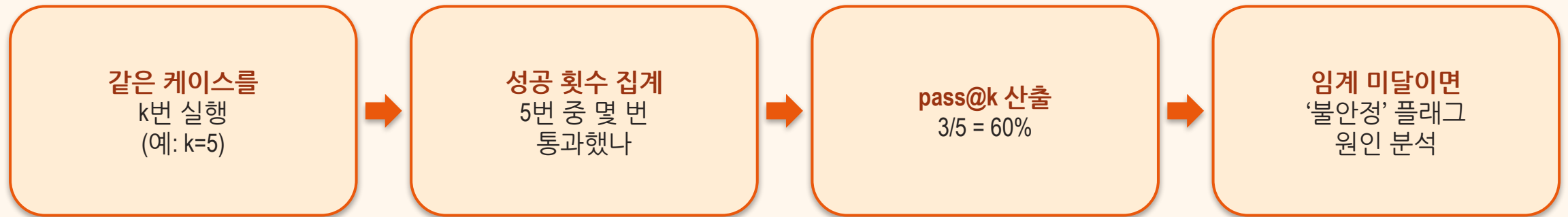
19.99만 환불해야 하는데 전액 환불? — 인자 검증

출력 구조

JSON 스키마·형식이 유지되는가

한 번 성공은 운 — k번 성공률이 실력 (pass@k)

▶ 불안정한 케이스는 '가끔 통과'로 숨는다 — 반복 실행으로 드러낸다



SECTION 03

Snapshot — 한 번 합격한 출력으로 변화를 잡는다

Snapshot 테스트 5단계

▶ 기대값을 일일이 쓰는 대신 '어제까지 좋았던 출력'을 기준으로

1

합격한 출력을 저장

한 번 검증한 결과를 '스냅샷'으로 보관

2

코드/프롬프트 변경

기능 추가·리팩터링·모델 교체

3

재실행 → ④ diff

새 출력을 스냅샷과 비교 (temperature=0)

4

판단

의도된 변화면 갱신 · 아니면 회귀

diff가 떴을 때

▶ 스냅샷 갱신 버튼은 편리한 만큼 가장 위험한 버튼

diff의 의미

“버그”가 아니라 “변화 알림”이다

의도된 개선

→ 스냅샷 갱신

의도치 않은 변화

→ 회귀로 처리

⚠ 무지성 갱신

회귀를 스냅샷 안에 묻어버린다 — 갱신은 의식적으로

스냅샷 diff, 이렇게 본다

▶ 전용 프레임워크 없이 difflib 몇 줄로 핵심을 데모

```
snapshot = llm_answer(PROMPT_V1)      # ① 합격 출력 저장
new_out  = llm_answer(PROMPT_V2)      # ② 변경 후 재실행
diff = difflib.unified_diff(
    snapshot.splitlines(), new_out.splitlines())
# ③ 변화가 있으면 사람이 판단: 개선인가 회귀인가
```

SECTION 04

AI 기반 디버깅 — 에러 전문을 컨텍스트로

AI 디버깅의 핵심은 '전문(전문)'

▶ 잘린 트레이스백은 잘린 진단을 부른다

무엇을 주나

에러 메시지 + 트레이스백 **전문** + 해당 코드 통째로

하지 말 것

요약해서 주기 — “뭔가 타입 에러가 나요” = 뭉툭한 진단

받는 것

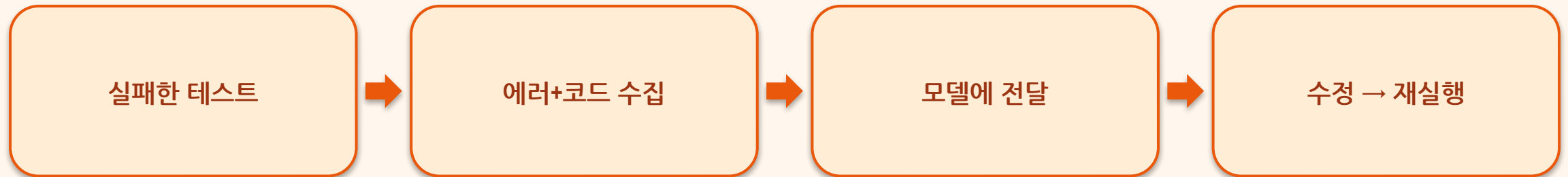
원인 추정 + 수정안

끝나는 조건

고쳤다고 끝이 아니라 **돌려서 확인**해야 끝 (day1 습관)

디버깅 루프

▶ 같은 모델이 만든 버그를 같은 모델이 못 볼 수도 있다 → 최종 심판은 테스트



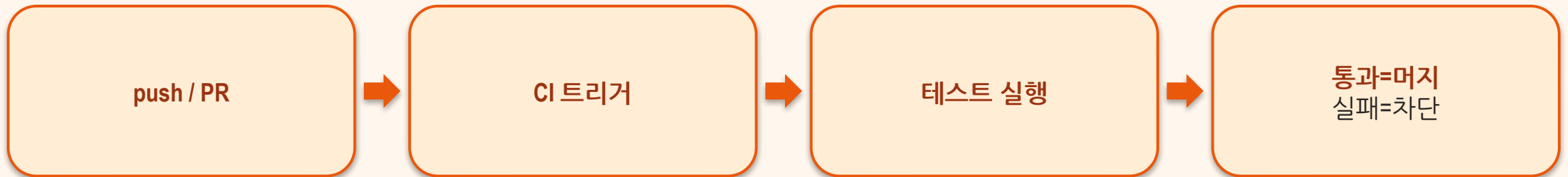
🔄 통과할 때까지 반복 — 수정 제안은 테스트로 재검증

SECTION 05

CI에 걸기 — 사람이 안 잊게 하는 장치

CI 흐름 한눈에

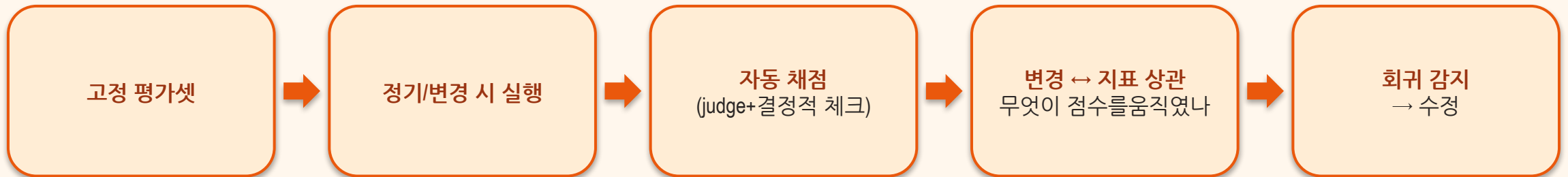
▶ “초록불일 때만 배포” — 회귀가 프로덕션에 못 가게 막는 문지기



🔔 실패면 알림 → 고쳐서 다시 push

평가 주도 루프 — 모델이 바뀌어도 잡는다

▶ 코드 머지만이 아니라 모델 업데이트·주기 실행으로도 트리거 — 드리프트 감지



🔗 Eval-Driven Development — 평가가 개발을 이끈다

자동화가 잡아야 할 ‘조용한 실패’ 4종

▶ 크래시 없이 틀리는 실패 — 각각 다른 감지 신호가 필요하다

환각

유창한 오답 — judge·근거 대조로 감지

프롬프트 회귀

사소한 수정 → 품질 하락 — 스냅샷·평가셋으로 감지

지연 스파이크

평균은 멀쩡한데 꼬리(p95/p99)가 폭발 — 성능 테스트로 감지

드리프트

입력·모델 분포가 서서히 변함 — 정기 실행 추이로 감지 (18차시)

자동화 + 사람, 둘 다

▶ 핵심 지표는 single source of truth 하나로 — 자동 평가 + 사람 샘플 검토 병행

자동화 — 매번 · 빠르게

- 머지·푸시마다 자동 실행
- 깨지면 합류 차단
- 대량 · 즉시 피드백

VS

사람 — 정기 · 깊게

- 샘플 검토 (human-in-the-loop)
- 미묘한 품질 문제 포착
- 자동화 사각지대의 보험

SECTION 06

정리

오늘 3줄

- 생성(위임) + 회귀(평가셋·스냅샷) + 디버깅(에러 전문) + CI(자동 실행) = AI 시대 테스트 루프
- LLM은 비결정적 → exact-match 대신 구조·의미 비교 + pass@k
- 테스트는 코드를, 사람은 테스트를 신뢰한다 — diff는 변화 알림

AI가 빨리 짤수록, 자동 검증이 더 중요해진다.

— 테스트는 코드를 검증하고, 그 테스트는 사람이 검증한다